Security Audit Report

MANTRA RWA Yield Distribution Smart Contract (claimdrop-v2-evm) (ID RWA-02)

v1.0

January 12, 2026

Table of Contents

Table of Contents 2
License 4
Disclaimer 5
Introduction 6
Purpose of This Report 6
Codebase Submitted for the Audit 6
Methodology 7
Functionality Overview 7
How to Read This Report 8
Code Quality Criteria 9
Summary of Findings 10
Detailed Findings 12
1. Missing old/new address equality guard lets allocations be wiped post-start 12
2. Unbounded campaign and distribution timelines allow it to last forever 12
3. removeAddress miscounts investors and allows duplicate entries in _investors 13
4. Closed campaigns permanently block pending claims 14
5. Lack of minimum distribution percentage allows dust entries and gas-griefing 14
6. Vesting schedules ignore campaign time bounds 15
7. Batch claim reverts when any recipient has already claimed 15
8. Missing cliff duration validation allows invalid vesting schedules 16
9. Campaigns can be created without sufficient funding, leading to claim failures 16
10. Unsafe downcast allows a silent overflow in claim amount tracking 17
11. addAllocations can exceed campaign.totalReward, or can remain shorter 18
12. The owner can close the campaign and refund early while the campaign is still active 18
13. Fee-on-transfer tokens are not handled in the claim balance check 19
14. amountClaimed can be increased to reduce the chance of overflow 19
15. Redundant zero-initialization wastes gas during campaign creation 20
16. Unpinned Solidity compiler version may lead to inconsistent bytecode across builds 20
17. Upgradable factory lacks a storage gap to protect future layout changes 21
18. Disabled factory reset creates test-only dependency on specific chain ID 21
19. Redundant distributions array deletion and inefficient assignment in createCampaign 22
20. The primarySale address cannot be updated once set in PrimarySaleClaimdropFactory 23
21. Import paths for OpenZeppelin dependencies are incorrect 23
22. Native token campaign is not supported 24
23. Redundant comparison in the dust handling condition 24

24. Lump sum distributions can have a non-zero cliffDuration 24
25. Ownable initializer signature mismatch with imported OpenZeppelin version 25
26. Unused receiptToken state variable 25
27. Blocklist status unnecessarily cleared on address replacement 26
28. getRewards function does not zero pending for blocklisted users 26

License

THIS WORK IS LICENSED UNDER A CREATIVE COMMONS ATTRIBUTION-NODERIVATIVES 4.0 INTERNATIONAL LICENSE.

Disclaimer

THE CONTENT OF THIS AUDIT REPORT IS PROVIDED “AS IS”, WITHOUT REPRESENTATIONS AND WARRANTIES OF ANY KIND.

THE AUTHOR AND HIS EMPLOYER DISCLAIM ANY LIABILITY FOR DAMAGE ARISING OUT OF, OR IN CONNECTION WITH, THIS AUDIT REPORT.

THIS AUDIT REPORT WAS PREPARED EXCLUSIVELY FOR AND IN THE INTEREST OF THE CLIENT AND SHALL NOT CONSTRUE ANY LEGAL RELATIONSHIP TOWARDS THIRD PARTIES. IN PARTICULAR, THE AUTHOR AND HIS EMPLOYER UNDERTAKE NO LIABILITY OR RESPONSIBILITY TOWARDS THIRD PARTIES AND PROVIDE NO WARRANTIES REGARDING THE FACTUAL ACCURACY OR COMPLETENESS OF THE AUDIT REPORT.

FOR THE AVOIDANCE OF DOUBT, NOTHING CONTAINED IN THIS AUDIT REPORT SHALL BE CONSTRUED TO IMPOSE ADDITIONAL OBLIGATIONS ON COMPANY, INCLUDING WITHOUT LIMITATION WARRANTIES OR LIABILITIES.

COPYRIGHT OF THIS REPORT REMAINS WITH THE AUTHOR.

This audit has been performed by

Oak Security GmbH

https://oaksecurity.io/
info@oaksecurity.io

Introduction

Purpose of This Report

Oak Security GmbH has been engaged by MANTRA Ventures Limited to perform a security audit of MANTRA RWA Yield Distribution Smart Contract (claimdrop-v2-evm) (ID RWA-02).

The objectives of the audit are as follows:

1. Determine the correct functioning of the protocol, in accordance with the project specification.
2. Determine possible vulnerabilities, which could be exploited by an attacker.
3. Determine smart contract bugs, which might lead to unexpected behavior.
4. Analyze whether best practices have been applied during development.
5. Make recommendations to improve code safety and readability.

This report represents a summary of the findings.

As with any code audit, there is a limit to which vulnerabilities can be found, and unexpected execution paths may still be possible. The author of this report does not guarantee complete coverage (see disclaimer).

Codebase Submitted for the Audit

The audit has been performed on the following target:

Repository	https://github.com/MANTRA-Chain/mantra-claimdrop-contract-v2
Commit	`b94158157f286e57bcab89838dbc020049c79957`
Scope	The scope is restricted to the Solidity smart contracts in the `evm/contracts` directory, excluding the smart contracts in `evm/contracts/mocks`.
Fixes verified at commit	`b493ec5231810eb25172c2a69bac10fbea237ae7` Note that only fixes to the issues described in this report have been reviewed at this commit. Any further changes such as additional features have not been reviewed.

Methodology

The audit has been performed in the following steps:

1. Gaining an understanding of the code base's intended purpose by reading the available documentation.
2. Automated source code and dependency analysis.
3. Manual line-by-line analysis of the source code for security vulnerabilities and use of best practice guidelines, including but not limited to:
    a. Race condition analysis
    b. Under-/overflow issues
    c. Key management vulnerabilities
4. Report preparation

Functionality Overview

MANTRA's claimdrop Solidity contracts enable configurable token distribution with flexible vesting and lump-sum schedules, optimized batch allocation processing, and robust administrative safeguards, including blocking certain addresses and emergency pausing.

How to Read This Report

This report classifies the issues found into the following severity categories:

Severity	Description
Critical	A serious and exploitable vulnerability that can lead to loss of funds, unrecoverable locked funds, or catastrophic denial of service.
Major	A vulnerability or bug that can affect the correct functioning of the system, lead to incorrect states or denial of service.
Minor	A violation of common best practices or incorrect usage of primitives, which may not currently have a major impact on security, but may do so in the future or introduce inefficiencies.
Informational	Comments and recommendations of design decisions or potential optimizations, that are not relevant to security. Their application may improve aspects, such as user experience or readability, but is not strictly necessary. This category may also include opinionated recommendations that the project team might not share.

The status of an issue can be one of the following: **Pending**, **Acknowledged**, **Partially Resolved**, or **Resolved**.

Note that audits are an important step to improving the security of smart contracts and can find many issues. However, auditing complex codebases has its limits and a remaining risk is present (see disclaimer).

Users of the system should exercise caution. In order to help with the evaluation of the remaining risk, we provide a measure of the following key indicators: **code complexity**, **code readability**, **level of documentation**, and **test coverage**. We include a table with these criteria below.

Note that high complexity or low test coverage does not necessarily equate to a higher risk, although certain bugs are more easily detected in unit testing than in a security audit and vice versa.

Code Quality Criteria

The auditor team assesses the codebase's code quality criteria as follows:

Criteria	Status	Comment
Code complexity	**Low-Medium**	-
Code readability and clarity	**High**	-
Level of documentation	**Medium-High**	-
Test coverage	**Medium-High**	`forge coverage --ir-minimum` reports a test line coverage of **87,62%**.

Summary of Findings

No	Description	Severity	Status
1	Missing old/new address equality guard lets allocations be wiped post-start	Minor	Resolved
2	Unbounded campaign and distribution timelines allow it to last forever	Minor	Resolved
3	`removeAddress` miscounts investors and allows duplicate entries in _investors	Minor	Resolved
4	Closed campaigns permanently block pending claims	Minor	Resolved
5	Lack of minimum distribution percentage allows dust entries and gas-griefing	Minor	Resolved
6	Vesting schedules ignore campaign time bounds	Minor	Resolved
7	Batch claim reverts when any recipient has already claimed	Minor	Resolved
8	Missing cliff duration validation allows invalid vesting schedules	Minor	Resolved
9	Campaigns can be created without sufficient funding, leading to claim failures	Minor	Resolved
10	Unsafe downcast allows a silent overflow in claim amount tracking	Minor	Resolved
11	`addAllocations` can exceed `campaign.totalReward`, or can remain shorter	Minor	Resolved
12	The owner can close the campaign and refund early while the campaign is still active	Minor	Resolved
13	Fee-on-transfer tokens are not handled in the claim balance check	Minor	Resolved
14	`amountClaimed` can be increased to reduce the chance of overflow	Informational	Acknowledged
15	Redundant zero-initialization wastes gas during campaign creation	Informational	Acknowledged
16	Unpinned Solidity compiler version may lead to inconsistent bytecode across builds	Informational	Resolved

17	Upgradable factory lacks a storage gap to protect future layout changes	**Informational**	**Resolved**
18	Disabled factory reset creates test-only dependency on specific chain ID	**Informational**	**Acknowledged**
19	Redundant `distributions` array deletion and inefficient assignment in `createCampaign`	**Informational**	**Resolved**
20	The `primarySale` address cannot be updated once set in `PrimarySaleClaimdropFactory`	**Informational**	**Acknowledged**
21	Import paths for OpenZeppelin dependencies are incorrect	**Informational**	**Acknowledged**
22	Native token campaign is not supported	**Informational**	**Resolved**
23	Redundant comparison in the dust handling condition	**Informational**	**Resolved**
24	Lump sum distributions can have a non-zero `cliffDuration`	**Informational**	**Resolved**
25	Ownable initializer signature mismatch with imported OpenZeppelin version	**Informational**	**Resolved**
26	Unused `receiptToken` state variable	**Informational**	**Acknowledged**
27	Blocklist status unnecessarily cleared on address replacement	**Informational**	**Resolved**
28	`getRewards` function does not zero pending for blocklisted users	**Informational**	**Resolved**

Detailed Findings

1. Missing old/new address equality guard lets allocations be wiped post-start

Severity: Minor

`evm/contracts/Claimdrop.sol:321-350` implements the `replaceAddress` function, allowing an authorized actor to migrate allocations, claims, and blocklist status from `oldAddress` to `newAddress`. The function validates that neither address is zero, that the old address has an allocation, and that the new address does not, but it never checks whether the two addresses are identical.

If `oldAddress == newAddress`, the loop in line `335` iterates over all distributions and first assigns each claim, then immediately deletes claims of the old address, effectively zeroing the caller's claims. In addition, the subsequent delete `allocations[oldAddress]` clears the allocation, and the blocklist flag is removed. Because `replaceAddress` lacks any pre-start restriction (unlike `removeAddress`, which guards `campaign.startTime` in lines `353-366`), this destructive call can be executed even after the campaign has begun, allowing an authorized account to erase an investor's allocation mid-campaign simply by passing the same address twice. This creates an unintended denial of funds risk for participants.

Recommendation

We recommend adding an explicit guard before performing the transfer to prevent self-replacement calls, ensuring allocations and claims cannot be zeroed by providing identical addresses.

Status: Resolved

2. Unbounded campaign and distribution timelines allow it to last forever

Severity: Minor

`evm/contracts/Claimdrop.sol:240-254` and the validation routine in `evm/contracts/Claimdrop.sol:542-566` only enforce that a campaign's `endTime` is later than `startTime`, and that each distribution's `start`/`end` ordering is consistent. There is no upper bound on `endTime` or on the duration of individual distributions.

As a result, an authorized caller can create a campaign that lasts decades (or effectively forever) and define linear-vesting slots that never finish. This traps user allocations in perpetuity, since rewards remain unclaimable until the vesting logic reaches completion.

Without a duration cap, misconfigurations or malicious operators can render the contract unusable for intended recipients and complicate token accounting, treasury planning, and off-chain auditing.

Recommendation

We recommend introducing explicit maximum durations (e.g., via a configurable constant) for both campaign lifetime (`endTime - startTime`) and each distribution's vesting window. Reject creations where either interval exceeds the allowed bound, so that rewards are guaranteed to unlock within a predictable timeframe.

Status: Resolved

3. `removeAddress` miscounts investors and allows duplicate entries in `_investors`

Severity: Minor

In `evm/contracts/Claimdrop.sol:355-366`, the `removeAddress` function deletes the allocation and `_hasAllocation` flag. Still, it never decrements the investor population tracked by `_investors`, so `getInvestorCount` can only increase even when wallets are removed.

Additionally, the function does not remove the address from the `_investors` array, so an address that is removed and later re-added through the `addAllocations` function will be pushed again, leading to duplicate entries and inflated counts.

Likewise, `replaceAddress` in `evm/contracts/Claimdrop.sol:321-351` transfers allocations and flags but does not update the `_investors` array, so the replaced address stays listed while the new address is never recorded. These inconsistencies break any accounting that depends on an accurate investor count or on iterating unique investors, e.g., distribution logic or analytics that rely on the `_investors` array length.

Recommendation

We recommend augmenting `removeAddress` to maintain explicit removal bookkeeping (e.g., decrement a dedicated counter or compact `_investors`) and ensure `getInvestorCount` subtracts removed wallets. Update both `removeAddress` and `replaceAddress` so the `_investors` array mirrors allocation ownership. If a removal occurs, replace the removed address with the last address in the list, then pop it. In the event of a replacement, replace the address with the new one.

Status: Resolved

4. Closed campaigns permanently block pending claims

Severity: Minor

In `evm/contracts/Claimdrop.sol:501-505`, the `_computePendingAmount` function, which is invoked when users query their eligible claim amount or actually claim funds, only returns a positive pending balance when `campaign.closedAt == 0`.

Once an admin closes a campaign, `closedAt` becomes non-zero, so the branch that calls `_computeClaimableAmount` is never executed, and pending is always set to 0. As a consequence, participants who accrued rewards before closure are permanently unable to claim them, even though closing should only stop new accrual rather than erase existing entitlements. This logic breaks expected campaign lifecycle guarantees and risks locking user funds.

Recommendation

We recommend adjusting the pending-amount logic to allow claims after closure by separating "campaign exists and has started" checks from the `closedAt` condition (e.g., compute the accrued amount up to `closedAt` or `block.timestamp`, whichever is earlier).

Status: Resolved

5. Lack of minimum distribution percentage allows dust entries and gas-griefing

Severity: Minor

The campaign validation logic in `evm/contracts/Claimdrop.sol:534-566` only checks that the distribution percentages sum up to `BASIS_POINTS_TOTAL`, but never enforces a per-distribution minimum.

A campaign creator can therefore append hundreds of distributions, each set to 0-1 basis points. Very small percentages cannot be represented accurately when converted to token amounts, so the campaign will leak value due to rounding and precision loss.

Additionally, large dust distributions inflate loop iterations across all claim paths, enabling griefers to raise gas costs or even cause a DoS for claimants if block gas limits are exceeded.

Recommendation

We recommend enforcing a reasonable lower bound (e.g., `≥10` or `≥100` bps) for each `Distribution.percentageBps` during `_validateCampaignParams`, and/or cap the total number of distributions. Reject configurations that would underflow to zero token amounts to avoid precision loss and remove the gas griefing vector.

Status: Resolved

6. Vesting schedules ignore campaign time bounds

Severity: Minor

`evm/contracts/Claimdrop.sol:528-565` validates campaign distributions but only checks that linear vesting `endTime` is greater than `startTime` (`evm/contracts/Claimdrop.sol:552-555`), and that the lump-sum `startTime` does not fall after the campaign `endTime` (`evm/contracts/Claimdrop.sol:557-560`).

There is no verification that a vesting distribution's `startTime` and `endTime` are contained within the campaign window. As a result, a linear vesting tranche can begin before `startTime` or finish after `endTime`, allowing claims to accrue outside the campaign's accounting period. This misalignment can cause underfunded campaigns, inaccurate total allocation tracking, and unexpected claim availability, especially if tokens continue to vest after the campaign nominally ends.

Recommendation

We recommend extending `_validateCampaignParams` to enforce that every linear vesting distribution satisfies `startTime >= campaign.startTime` and `endTime <= campaign.endTime`, reverting otherwise. This keeps vesting schedules aligned with the campaign lifecycle and prevents allocations from being claimable outside the intended window.

Status: Resolved

7. Batch claim reverts when any recipient has already claimed

Severity: Minor

`evm/contracts/Claimdrop.sol:382-402` processes admin-triggered batch claims by iterating through users and calling `_claim` for each entry. The underlying `_claim` path strictly reverts when an address has no remaining claims.

For example, when `allocation == 0`, it throws the `NoAllocation` error. Because the loop does not isolate failures, the entire TX reverts if any user in the batch was frontrun and already claimed, even if the rest of the batch is valid. A malicious participant can grief authorized operators by pre-claiming a minimal amount (e.g., one wei) and forcing every administrative batch transaction that includes them to fail until they are removed from calldata in case non-zero amounts are used for the users.

This denial-of-service prevents the timely distribution of rewards to other recipients and undermines the purpose of the batch-claim feature.

Recommendation

We recommend making `_claim` tolerant in the batch context, e.g., skip users with zero remaining allocation (emit an event and continue) or allow a zero amount to signify "claim full balance without reverting.".

Additionally, consider handling claims that would exceed the accrued limit gracefully, i.e., claim `min(accrued, requested)`. This ensures a malicious or already-claimed entry cannot block the entire administrative batch.

Status: Resolved

8. Missing cliff duration validation allows invalid vesting schedules

Severity: Minor

In `evm/contracts/Claimdrop.sol:543-567`, the `_validateCampaignParams` function validates the distribution percentages and time bounds, but never checks that a linear vesting distribution's `cliffDuration` is shorter than the overall vesting period (`endTime - startTime`). As a result, configurations where `startTime + cliffDuration >= endTime` pass validation even though nothing should vest during the cliff.

Such a setup collapses the vesting logic into a delayed lump-sum release, i.e., 100% of tokens unlock immediately after the cliff, contradicting the intended gradual vesting behavior. If misconfigured, campaign administrators could unintentionally create distributions that bypass vesting altogether, undermining investor expectations and token release guarantees.

Recommendation

We recommend extending the validation loop in `evm/contracts/Claimdrop.sol:543-567` to enforce `cliffDuration < endTime - startTime` (or equivalently `distributions[i].startTime + cliffDuration <= distributions[i].endTime`) for all linear vesting entries, reverting with an explicit error when the cliff would outlast the vesting window.

Status: Resolved

9. Campaigns can be created without sufficient funding, leading to claim failures

Severity: Minor

In `contracts/Claimdrop.sol:211-254`, the `createCampaign` function initializes a new campaign with a specified `totalReward` amount. However, the function does not verify whether the contract holds sufficient `rewardToken` to cover this liability, nor does it attempt

to transfer tokens from the creator. This design uses a "push" funding pattern, where the creator is expected to manually transfer funds to the contract after creation.

If the campaign creator fails to manually transfer the required `rewardToken` amount to the contract, users attempting to claim their allocated rewards will be unable to do so due to insufficient funds.

Recommendation

We recommend implementing a "pull" mechanism in the `createCampaign` function to transfer the `totalReward` amount from the `msg.sender` to the contract at the time of campaign creation.

Alternatively, enforce a check that `IERC20(rewardToken).balanceOf(address(this))` is at least equal to `totalReward` before allowing the campaign to be created.

Status: Resolved

10. Unsafe downcast allows a silent overflow in claim amount tracking

Severity: Minor

In `contracts/Claimdrop.sol:820`, the `claim` function updates the `amountClaimed` value in the `Claim` struct. It involves adding the current distribution amount to the previously claimed amount and casting the result to `uint128`.

Since explicit casting in Solidity does not check for overflows, if the sum exceeds the maximum value of `uint128`, the value will be silently truncated. This would lead to an incorrect `amountClaimed` state, potentially allowing users to claim more tokens than intended or disrupting the accounting of distributed rewards.

We classify this issue as minor because it requires an unrealistically large number of tokens to overflow the `uint128` limit, assuming the reward token has reasonable decimals and a total supply.

Recommendation

We recommend using a safe casting library, such as OpenZeppelin's `SafeCast`, or manually validating that the value fits within `uint128` before casting.

Status: Resolved

11. `addAllocations` can exceed `campaign.totalReward`, or can remain shorter

Severity: Minor

In `evm/contracts/Claimdrop.sol:277-315`, `addAllocations` loops over the provided `addresses` and `amounts`, updates `allocations[addr]`, and accumulates `totalAmount += amount`, but never checks whether the sum of all allocations stays within `campaign.totalReward`. This allows an authorized admin to accidentally over-allocate rewards beyond the configured campaign total, so the contract can promise more tokens than it actually holds or was configured to distribute.

Recommendation

We recommend adding a validation that the new total allocated amount (including any existing allocations) does not exceed `campaign.totalReward`, and reverting if the additions would oversubscribe the campaign.

Status: Resolved

12. The owner can close the campaign and refund early while the campaign is still active

Severity: Minor

In `evm/contracts/Claimdrop.sol:256-272`, the `closeCampaign` function only checks `campaign.exists` and `campaign.closedAt != 0`, but never verifies that `block.timestamp >= campaign.endTime`, so the owner can close the campaign and sweep all remaining tokens at any time, even while vesting is still active. This might be intended as an emergency stop, but in practice it allows the owner to rug active campaigns by ending them early and reclaiming unclaimed rewards.

Recommendation

We recommend adding a require or custom error to enforce `block.timestamp >= campaign.endTime` before allowing `closeCampaign`, or clearly documenting and gating the early-close behavior if early refunds are intentionally allowed.

Status: Resolved

13. Fee-on-transfer tokens are not handled in the claim balance check

Severity: Minor

In `evm/contracts/Claimdrop.sol:807-809` and `835-837`, the `_claim` function first checks `contractBalance = IERC20(campaign.rewardToken).balanceOf(address(this))` and reverts `if contractBalance < claimAmount`, then later calls `safeTransfer(receiver, claimAmount)` assuming a 1:1 transfer.

For fee-on-transfer tokens, the contract's pre-transfer balance may be sufficient, but the amount delivered to the receiver will be less than `claimAmount`, so users' allocation accounting will be updated as if they received the full amount, even though they actually received less.

Recommendation

We recommend adapting the claim flow to support fee-on-transfer tokens by measuring balances before and after the transfer and using the actual delta as the effective claimed amount, or by explicitly documenting and enforcing that only standard, non-FOT ERC20 tokens are supported for `campaign.rewardToken`.

Status: Resolved

14. `amountClaimed` can be increased to reduce the chance of overflow

Severity: Informational

In `evm/contracts/Claimdrop.sol:89-93`, the `Claim` struct stores `amountClaimed` as `uint128` along with a `uint64` timestamp in the same storage slot. Because the struct still leaves 64 unused bits inside that slot, the counter is limited to 128 bits even though campaign rewards, allocation accounting, and ERC20 balances elsewhere in the contract use 256-bit math.

If the cumulative amount claimed for a distribution slot ever exceeds `2^128 - 1`, the field will wrap, allowing users to continue claiming despite having already exhausted their allocation, corrupting the slot's accounting. While such an overflow is unlikely, it is unnecessary because a larger integer width can be used without increasing gas or storage costs.

Recommendation

We recommend increasing `amountClaimed` to `uint192` (or another wider type that still keeps the struct within a single 32-byte slot). Hence, the field better matches the magnitude

of campaign rewards and further reduces the risk of overflow while preserving storage efficiency.

Status: Acknowledged

The client acknowledges this finding with the note that the amounts involved will not be large enough to risk overflow.

15. Redundant zero-initialization wastes gas during campaign creation

Severity: Informational

In `evm/contracts/Claimdrop.sol:234-245`, the `createCampaign` function assigns `campaign.claimed = 0` and `campaign.closedAt = 0` immediately after populating other campaign fields. Because both claimed and closedAt are storage variables of a fresh struct, Solidity already initializes them to zero. Writing explicit zeros therefore consumes additional gas without changing state, making campaign creation slightly more expensive and setting an unnecessary pattern for future modifications.

Recommendation

We recommend removing the redundant assignments to `campaign.claimed` and `campaign.closedAt` in `createCampaign` and relying on Solidity's default zero-initialization for new storage structs.

Status: Acknowledged

The client acknowledges this finding with the note that the low cost of gas on the MANTRA Chain ensures the operation remains affordable despite the identified gas inefficiency.

16. Unpinned Solidity compiler version may lead to inconsistent bytecode across builds

Severity: Informational

Both `Claimdrop` and `PrimarySaleClaimdropFactory` declare pragma solidity `^0.8.24`, allowing any compiler version from `0.8.24` up to—but not including—`0.9.0`. This floating pragma means different environments may compile the contracts with different patch-level compilers, each potentially emitting slightly different bytecode or enabling/disabling compiler-level bug fixes. Such variability complicates deterministic builds, auditing, and reproducibility of deployed code, and could introduce latent bugs if future compiler releases change semantics or optimize code differently. Without a pinned compiler, audits performed against one binary may not match the bytecode actually deployed later.

Recommendation

We recommend pinning the compiler to an exact version (e.g., `0.8.24`).

Status: Resolved

17. Upgradable factory lacks a storage gap to protect future layout changes

Severity: Informational

`evm/contracts/PrimarySaleClaimdropFactory.sol:15-254` defines the `PrimarySaleClaimdropFactory` contract as an upgradeable contract by inheriting `Initializable`, `OwnableUpgradeable`, and `PausableUpgradeable`, but the storage layout concludes in line `253` without reserving a private `__gap` array.

OpenZeppelin's upgradeable design guidelines require a storage gap at the end of each upgradeable implementation to allow new state variables to be appended in later versions without shifting inherited storage slots.

Without this reserve, any future version that adds state variables risks overwriting existing slots (for example, corrupting `deployedClaimdrops`, `isClaimdrop`, or `primarySale`), which can permanently brick the proxy or allow privilege escalation.

This vulnerability is particularly severe because the factory controls the deployment of downstream claim drops and primary sale addresses, so state corruption can halt deployments or misconfigure critical addresses.

Recommendation

We recommend appending a `uint256[43] private __gap` (50 slots - 7 slots that are already used) at the end of `PrimarySaleClaimdropFactory` and ensuring all upgradeable contracts in the project follow the OpenZeppelin storage-gap pattern to preserve storage-layout compatibility across upgrades.

Status: Resolved

18. Disabled factory reset creates test-only dependency on specific chain ID

Severity: Informational

In `evm/contracts/PrimarySaleClaimdropFactory.sol:188-207`, the `resetFactory` function prevents execution when `block.chainid == 5888`, effectively hard-coding MANTRA Mainnet as non-resettable. This implementation addresses a production safety concern but also requires that any testing or redeployment on other live

networks rely on the same chain restriction. If the factory contract is deployed to a different production chain that still requires the reset safeguard, the logic becomes unreliable—tests must either fork chain `5888` or modify production code, and deployments to other mainnets could omit the intended restriction. Relying on an in-contract chain check ties testing behavior to a specific chain ID and risks shipping chain-specific logic that may be forgotten or misapplied when the factory is reused elsewhere.

Recommendation

We recommend moving the `resetFactory` functionality into a dedicated test-only subclass deployed only in the test environment. This keeps `PrimarySaleClaimdropFactory` chain-agnostic while allowing tests to exercise reset behavior without depending on chain IDs.

Status: Acknowledged

The client acknowledges this finding with the note that this functionality is used only for internal testing.

19. Redundant `distributions` array deletion and inefficient assignment in `createCampaign`

Severity: Informational

In `contracts/Claimdrop.sol:248`, the `createCampaign` function deletes the `campaign.distributions` array before populating it. Since the function enforces that `campaign.exists` is `false`, the `campaign` struct is in an uninitialized state, with the `distributions` array already empty. Consequently, the `delete` operation is redundant and consumes unnecessary gas.

Additionally, the loop in lines `249-251` populates the `campaign.distributions` array by pushing elements individually from the `distributions` calldata array. This approach is less gas-efficient and more verbose than a direct array assignment.

Recommendation

We recommend removing the `delete` statement and assigning the distributions directly using `campaign.distributions = distributions;` to improve gas efficiency and code clarity.

Status: Resolved

20. The `primarySale` address cannot be updated once set in `PrimarySaleClaimdropFactory`

Severity: Informational

In `contracts/PrimarySaleClaimdropFactory.sol:127`, the `setPrimarySale` function enforces that the `primarySale` address can only be set when it is currently `address(0)`.

This check prevents the factory owner from updating the `primarySale` address after the initial configuration. If an incorrect address is set or if the `PrimarySale` contract needs to be rotated, the owner cannot update the reference.

Recommendation

We recommend allowing the owner to update the `primarySale` address even if it is already set, or documenting the immutability of this address clearly.

Status: Acknowledged

The client acknowledges this finding with the note that it is an intentional design choice for one factory to link to only one `PrimarySale` contract.

21. Import paths for OpenZeppelin dependencies are incorrect

Severity: Informational

In `evm/contracts/PrimarySaleClaimdropFactory.sol:4-6`, the import path for the `PausableUpgradeable` contract uses `@openzeppelin/contracts-upgradeable/utils/PausableUpgradeable.sol`, but in the standard OpenZeppelin source layout, `PausableUpgradeable` is found in the `security` folder, not `utils`. Unless there is a non-standard remapping that redirects this path, this will cause compilation errors or make the project dependent on a custom local setup.

Recommendation

We recommend changing the import to `@openzeppelin/contracts-upgradeable/security/PausableUpgradeable.sol` (or adjusting the remappings) so it matches the official OpenZeppelin folder structure and compiles reliably across environments.

Status: Acknowledged

The client acknowledges this finding, noting that the project builds successfully in their environment, indicating that the current import paths resolve correctly within their build configuration.

22. Native token campaign is not supported

Severity: Informational

In `evm/contracts/Claimdrop.sol:227`, the `createCampaign` function enforces `if (rewardToken == address(0)) revert ZeroAddress();`, which means this contract cannot be used to distribute native tokens using the common pattern of treating `address(0)` as the native asset.

Any attempt to set up a campaign with `rewardToken == address(0)` to represent native rewards will just revert at creation time instead of working, so if the intention was to support native token distributions later on, this check quietly prevents that flow.

Recommendation

We recommend either making it explicit in the documentation and tests that `Claimdrop` only supports ERC20 reward tokens and never native tokens, or, if native distributions are required, redesigning the `rewardToken` handling so `address(0)` is allowed as a native token sentinel and is properly handled in funding and claim logic instead of being rejected by `ZeroAddress`.

Status: Resolved

23. Redundant comparison in the dust handling condition

Severity: Informational

In `evm/contracts/Claimdrop.sol:615`, the dust handling logic uses `if (expectedRemaining > claimable && expectedRemaining - claimable > 0)`, but `expectedRemaining - claimable > 0` is strictly implied by `expectedRemaining > claimable` for unsigned integers, so the second part of the condition is redundant and will never change the branch behavior.

Recommendation

We recommend simplifying the condition to `if (expectedRemaining > claimable)` to keep the dust adjustment logic clearer and avoid redundant checks.

Status: Resolved

24. Lump sum distributions can have a non-zero `cliffDuration`

Severity: Informational

In `evm/contracts/Claimdrop.sol:558`, `_validateCampaignParams` only checks that LumpSum distributions satisfy `distributions[i].startTime > endTime`, but it

never enforces that `distributions[i].cliffDuration` is zero when `kind == DistributionKind.LumpSum`.

Recommendation

We recommend extending the LumpSum validation branch to also require `distributions[i].cliffDuration == 0` for `DistributionKind.LumpSum`, so the configuration surface stays clean and there is no ambiguity around `cliffDuration` in one-off LumpSum distributions.

Status: Resolved

25. Ownable initializer signature mismatch with imported OpenZeppelin version

Severity: Informational

In `evm/contracts/PrimarySaleClaimdropFactory.sol:93`, the `initializer` calls `__Ownable_init(initialOwner);`, but the imported OpenZeppelin `OwnableUpgradeable` (v4.9.0 in `evm/lib/openzeppelin-contracts-upgradeable/contracts/access/OwnableUpgradeable.sol:29-35`) only exposes a zero-argument function `__Ownable_init()` that sets the owner to `_msgSender()`. With the current OpenZeppelin version and remappings, there is no `__Ownable_init(address)` overload, so this code will not compile unless the import or library version is changed to one that supports the address-argument initializer.

Recommendation

We recommend aligning the initializer with the actual `OwnableUpgradeable` version in use, either by updating the OpenZeppelin upgradeable contracts to a version that provides `__Ownable_init(address)` or by changing the factory initializer to call the zero-argument `__Ownable_init()` and then explicitly transferring ownership to `initialOwner` (or otherwise documenting that the deployer becomes the owner).

Status: Resolved

26. Unused `receiptToken` state variable

Severity: Informational

In `evm/contracts/PrimarySaleClaimdropFactory.sol:33-37` and `200-204`, the `receiptToken` state variable is declared and only ever written in `resetFactory`, where it is reset to `address(0)`. It is never read or otherwise used by the factory logic, so it is currently a dead field that only increases the storage layout surface.

Recommendation

We recommend either removing `receiptToken` entirely if it is not needed, or adding a clear NATSPEC comment and wiring that explains how and where `receiptToken` is expected to be set and used so that the storage slot and reset behavior are justified.

Status: Acknowledged

The client acknowledges this finding with the note that the unused `receiptToken` state variable is intentionally retained for future usage.

27. Blocklist status unnecessarily cleared on address replacement

Severity: Informational

In `evm/contracts/Claimdrop.sol:343`, the `replaceAddress` function copies the blocklist flag from `oldAddress` to `newAddress`, but then deletes `blacklist[oldAddress]`, so the original address is no longer blocklisted after the migration.

If the intention is for a blocklisted user to remain blocked regardless of which address they use, this behavior weakens the blocklist by unblocking the old address while keeping only the new one blocklisted.

Recommendation

We recommend keeping `blacklist[oldAddress]` set to `true` when transferring the flag, by removing the `delete blacklist[oldAddress]` line so that both the old and new addresses are blocklisted after replacement, or otherwise clearly document that replacing an address is intended to remove the old address from the blocklist.

Status: Resolved

28. `getRewards` function does not zero pending for blocklisted users

Severity: Informational

In `evm/contracts/Claimdrop.sol:497-505`, the `getRewards` function computes pending rewards via `_computeClaimableAmount(addr, total)` with no blocklist check. This means a blocklisted address will still see a positive pending amount returned from `getRewards`, even though actual claiming is blocked elsewhere.

Recommendation

We recommend extending the condition to also check `!blacklist[addr]` (or explicitly setting `pending = 0` when `blacklist[addr]` is `true`) so that blocklisted users see zero pending rewards from `getRewards` in line with the on-chain claim restrictions.

Status: Resolved